

SPipelines: Complete Architecture, Files & Attributes Specification

This document provides an exhaustive, component-by-component architectural specification of **every file inside the [SPipelines/](#) folder**. For each file, this guide details its **exact operational purpose**, the **files it references (inputs)**, the **files it creates (outputs)**, and every **attribute, column, or feature it generates**.

1. Directory Structure Overview

SPipelines/	
■■■■ .env	<- Database connection string
■■■■ .gitattributes	<- Git LFS and text attributes
■■■■ all_trains.json	<- Raw NTES dump of all trains
■■■■ AppServer.py	<- FastAPI backend application server
■■■■ BlackSheep_Architecture_and_Files_Summary.pdf	<- Architecture executive summary
■■■■ Pipelines_Architecture_Specification.pdf	<- Comprehensive pipeline PDF guide
■■■■ requirements.txt	<- Python package dependencies
■■■■ SetupNeonRelationalSchema.py	<- Neon PostgreSQL DDL relational schema setup
■■■■ SheepBand.py	<- Live calibrated EA engine & scenario extractor
■■■■ UploadAllToNeon.py	<- Master database migration and uploader
■	
■■■■ Frontend/	<- Live interactive web UI
■ ■■■■ app.css	<- Stylesheet (glassmorphism, dark/light theme)
■ ■■■■ app.js	<- Client-side REST consumer & scenario builder
■ ■■■■ index.html	<- Main dashboard layout & simulation controls
■	
■■■■ Pipelines/	<- Cascading delay simulation pipeline
■ ■■■■ 01Preprocess.py	<- Data cleaner & base EA slack calculator
■ ■■■■ 01bKineticAccelDecelDelay.py	<- Davis drag kinetic deceleration/acceleration model
■ ■■■■ 05CascadeBlockHeadway.py	<- Cascading Stage 1: Block section headway queuing
■ ■■■■ 06CascadeJunctionStarvation.py	<- Cascading Stage 2: Junction platform starvation
■ ■■■■ 07CascadeTurnaroundLinkage.py	<- Cascading Stage 3: Rake turnaround linkage
■ ■■■■ 08CascadeCrossingConflicts.py	<- Cascading Stage 4: Single-track crossing conflicts
■ ■■■■ 09CascadeCrewDutyExpiry.py	<- Cascading Stage 5: HOER crew duty timeout
■ ■■■■ GenerateMasterPnC.py	<- 38,880-scenario master PnC matrix builder
■ ■■■■ GenerateTRETARoutes.py	<- TRETA route number and zone partitioning
■ ■■■■ MLCompoundDelayModel.py	<- Multi-domain ML compound simulation engine
■ ■■■■ Orchestrar.py	<- Master pipeline orchestrator & ETA synthesizer
■ ■■■■ __init__.py	<- Python package initialization marker
■	
■■■■ Train-Data/	<- Base datasets, caches, and pipeline outputs
■ ■■■■ TrainsMetadata_DND.csv	<- Irreplaceable raw train metadata
■ ■■■■ TrainSchedules_DND.csv	<- Irreplaceable raw timetable stops and halts
■ ■■■■ EACalculation.csv	<- Preprocessed base metrics & calibrated EA
■ ■■■■ SlowAccDelay.csv	<- Kinetic stopping & acceleration profiles
■ ■■■■ HeadwayDelay.csv	<- Stage 1 headway simulation results
■ ■■■■ PlatformAllocDelay.csv	<- Stage 2 platform starvation results
■ ■■■■ ServicingDelay.csv	<- Stage 3 turnaround linkage results
■ ■■■■ SingleTrackDelay.csv	<- Stage 4 crossing loop results
■ ■■■■ TiredCrewDelay.csv	<- Stage 5 crew duty timeout results
■ ■■■■ FinalOrchestraOutput.csv	<- Consolidated master prediction output
■ ■■■■ masterPnC.csv	<- 38,880 unique disturbance permutations
■ ■■■■ PnCOutput.csv	<- 477,900 fleet scenario permutations
■ ■■■■ MLCompoundPermutations.csv	<- Multi-domain ML permutation dataset
■ ■■■■ TRETARoutes.csv	<- Canonical route ID mapping
■ ■■■■ TRETARouteElements.csv	<- 200,126 station-hop route zone blocks
■	
■■■■ documentation/	<- Engineering guides and documentation
■■■■ AboutCodebase-Implementation.md	<- Implementation notes
■■■■ AccDecKinetics.md	<- Mechanical kinetics documentation
■■■■ Cascading-DelayPlan.md	<- Cascade architecture strategy
■■■■ DeleteMe.md	<- Guide to deletable intermediate artifacts
■■■■ Pipelines_Architecture_Specification.md	<- Complete pipeline architecture markdown
■■■■ README.md	<- Repository overview

2. Root Application & Server Files

2.1. AppServer .py

- **File Path:** `SPipelines/AppServer.py`
- **Exact Role & Purpose:**

Production FastAPI server powering the web application and REST API endpoints.

- Loads high-speed in-memory caches from `Train-Data/` on startup for sub-millisecond query responses.
- Exposes REST endpoints:
- `GET /api/status`: System health check and dataset counts.
- `GET /api/predict`: Real-time delay evaluation across trains, weather, congestion, and TSR.
- `POST /api/predict/compound`: Complex multi-parameter disturbance simulation incorporating TRETA route blocks, alarm chain pulling, and engine failures.
- Mounts static files from `Frontend/` and serves `index.html`.
- **Files It References (Inputs):**
- `Train-Data/EACalculation.csv` (Fast metadata and calibrated EA cache)
- `Train-Data/PnCOutput.csv` (45 scenario permutations)
- `Train-Data/SlowAccDelay.csv` (Kinetic profile cache)
- `Train-Data/TrainSchedules_DND.csv` (Destination timetable arrival times)
- `Frontend/` directory (Static web assets)
- Imports: `SheepBand.py` (`execute_sheepband_orchestration`)
- **Files It Creates (Outputs):**
- Serves live JSON HTTP responses and web pages (does not write disk files).
- **Attributes It Makes / Exposes:**

Attribute Name	Data Type	Description
<code>train_no</code>	String	5-digit zero-padded train number (e.g. <code>'12001'</code>).
<code>train_name</code>	String	Commercial train name (e.g. <code>'NDLS SHATABDI'</code>).
<code>train_tier</code>	String	Priority tier (<code>T1_PREMIUM</code> , <code>T2_SUPERFAST</code> , <code>T3_EXPRESS</code>).
<code>treta_route_number</code>	String	Canonical corridor identifier (e.g. <code>'TRETA-NDLS-BPL-Z10-12001'</code>).
<code>primary_breakdown</code>	Object	Dictionary of primary shock metrics (<code>d_weather</code> , <code>d_tsr</code> , <code>d_accel_decel</code> , <code>d_chain_pulling</code> , <code>d_engine_failure</code> , <code>PrimaryDelay</code>).
<code>cascade_breakdown</code>	Object	Dictionary of network cascading metrics (<code>d_headway</code> , <code>d_junction</code> , <code>d_turnaround</code> , <code>d_crossing</code> , <code>d_crew</code> , <code>CascadeDelay</code>).
<code>math_resolution</code>	Object	Compounding mathematical breakdown: <code>grossDelay</code> , <code>EA_allotted_mins</code> , <code>Absorbed_by_EA</code> , <code>NetDelay</code> , <code>ScheduledDestinationArrival</code> , <code>Predicted_ETA</code> , <code>Arrival_Status</code> .

2.2. SheepBand .py

- **File Path:** `SPipelines/SheepBand.py`
- **Exact Role & Purpose:**

The core business logic and orchestrator extraction engine.

- Takes operational scenario parameters submitted from the frontend or API.
- Queries the exact disturbance row from the Neon PostgreSQL **master_pnc** table (falling back to local **Train-Data/masterPnC.csv**).
- Injects the estimated disturbances into the train's calibrated Extra Time (**EA**) formula.
- Translates net delay minutes into real-world clock arrival times (e.g., **23:30 (Day 1)**).
- **Files It References (Inputs):**
 - **Train-Data/EACalculation.csv** (Train metadata and calibrated slack)
 - **Train-Data/TrainSchedules_DND.csv** (Timetable arrival stops)
 - **Train-Data/TRETARoutes.csv** (Route numbers)
 - **Train-Data/masterPnC.csv** (Offline fallback matrix)
- Remote Neon PostgreSQL Table: **master_pnc**
- **Files It Creates (Outputs):**
 - In-memory orchestration result dictionaries.
- **Attributes It Makes:**

Attribute Name	Data Type	Description
grossDelay	Float	PrimaryDelay + CascadeDelay.
Absorbed_by_EA	Float	Delay absorbed by timetable slack: min(grossDelay recovery_rate, EA_allotted 0.50) .
NetDelay	Float	Final unrecovered delay: max(0.0, grossDelay - Absorbed_by_EA) .
Predicted_ETA	String	Real-world predicted clock arrival time (e.g. ' 23:45 (Day 1) ').
Arrival_Status	String	' ON_TIME ' if NetDelay <= 5.0 mins, else ' LATE '.

2.3. SetupNeonRelationalSchema.py

- **File Path:** **SPipelines/SetupNeonRelationalSchema.py**
- **Exact Role & Purpose:**

Database DDL migration script that enforces relational integrity and constraints across Neon PostgreSQL:

- Establishes **PRIMARY KEY** on parent master table **trains_metadata (train_no)**.
- Establishes 1-to-1 **PRIMARY KEY** and **FOREIGN KEY** constraints on cascade pipeline tables.
- Creates composite primary keys on 1-to-many child tables (**train_halts**, **delay_predictions**).
- Builds composite performance indexes (**idx_delay_lookup**, **idx_treta_routes_route_num**, etc.).
- **Files It References (Inputs):**
 - Remote Neon PostgreSQL database via connection string in **.env**.
- **Files It Creates (Outputs):**
 - Relational schema, tables, constraints, foreign keys, and indexes in Neon PostgreSQL.
- **Attributes It Makes / Enforces:**
 - **pk_trains_metadata:** Primary key constraint on **trains_metadata(train_no)**.
 - Foreign key cascades on: **ea_calculation**, **slow_acc_delay**, **headway_delay**, **platform_alloc_delay**, **servicing_delay**, **single_track_delay**, **tired_crew_delay**, **final_orchestra_output**, **treta_routes**, **train_halts**, **delay_predictions**, **treta_zone_elements**.

2.4. UploadAllToNeon.py

- **File Path:** `SPipelines/UploadAllToNeon.py`
- **Exact Role & Purpose:**

The production data loader and synchronization pipeline for the remote database.

- Reads all 15 project CSV datasets in strict relational dependency order (parent master table first, followed by children).
- Cleans column names to strictly lowercase and standardizes train numbers to 5-digit zero-padded strings.
- Performs batch uploads using `chunksize=5000` with `method='multi'` to eliminate SSL network timeouts.
- **Files It References (Inputs):**
 - `Train-Data/TrainsMetadata_DND.csv` $\rightarrow$ table `trains_metadata`
 - `Train-Data/TRETARoutes.csv` $\rightarrow$ table `treta_routes`
 - `Train-Data/EACalculation.csv` $\rightarrow$ table `ea_calculation`
 - `Train-Data/SlowAccDelay.csv` $\rightarrow$ table `slow_acc_delay`
 - `Train-Data/HeadwayDelay.csv` $\rightarrow$ table `headway_delay`
 - `Train-Data/PlatformAllocDelay.csv` $\rightarrow$ table `platform_alloc_delay`
 - `Train-Data/ServicingDelay.csv` $\rightarrow$ table `servicing_delay`
 - `Train-Data/SingleTrackDelay.csv` $\rightarrow$ table `single_track_delay`
 - `Train-Data/TiredCrewDelay.csv` $\rightarrow$ table `tired_crew_delay`
 - `Train-Data/FinalOrchestraOutput.csv` $\rightarrow$ table `final_orchestra_output`
 - `Train-Data/TrainSchedules_DND.csv` $\rightarrow$ table `train_halts`
 - `Train-Data/TRETAZoneElements.csv` $\rightarrow$ table `treta_zone_elements`
 - `Train-Data/MLCompoundPermutations.csv` $\rightarrow$ table `ml_compound_permutations`
 - `Train-Data/PnCOutput.csv` $\rightarrow$ table `delay_predictions`
 - `Train-Data/masterPnC.csv` $\rightarrow$ table `master_pnc`
- **Files It Creates (Outputs):**
 - Fully populated relational database in Neon PostgreSQL across all 15 tables.
- **Attributes It Makes / Formats:**
 - Normalizes all column headers to lowercase underscores (`train_no`, `duration_mins`, `net_slack_mins`, etc.).

2.5. Root Data, Config & PDF Files

- `.env`: Stores the active Neon PostgreSQL SSL connection URL (`POSTGRESQL="..."`).
 - `.gitattributes`: Configures Git Large File Storage (LFS) and line ending rules.
 - `requirements.txt`: Minimal production dependencies: `fastapi`, `uvicorn`, `pandas`, `numpy`, `sqlalchemy`, `psycpg2-binary`, `reportlab`.
 - `all_trains.json`: 52.7 MB raw NTES JSON dump containing raw train schedules.
 - `BlackSheepArchitectureandFilesSummary.pdf`: Initial architecture summary PDF.
 - `PipelinesArchitectureSpecification.pdf`: 14-page publication-grade PDF specification generated by `ToPDF.py`.
-

3. Pipelines Modules (SPipelines/Pipelines/)

3.1. 01Preprocess.py

- **File Path:** `SPipelines/Pipelines/01Preprocess.py`
- **Exact Role & Purpose:**

Base data ingestion and feature engineering engine. Parses raw travel time and halt duration strings, aggregates station dwells, assigns operational tiers (`T1_PREMIUM`, `T2_SUPERFAST`, `T3_EXPRESS`), and derives physical cruising times, kinetic braking overhead, and the calibrated **Net Slack / Extra Time (EA)**.

- **Files It References (Inputs):**
 - `Train-Data/TrainsMetadata_DND.csv` (Raw train metadata)
 - `Train-Data/TrainSchedules_DND.csv` (Raw halt records)
- **Files It Creates (Outputs):**
 - `Train-Data/EACalculation.csv` (Master base preprocessed fleet dataset)
- **Attributes It Makes:**
 - `duration_mins`: Journey duration in minutes.
 - `halt_mins`: Station dwell time in minutes.
 - `total_halt_count`: Number of scheduled stops.
 - `total_halt_duration`: Cumulative scheduled halt duration.
 - `total_distance_km`: Route distance in kilometers.
 - `train_tier`: `T1_PREMIUM`, `T2_SUPERFAST`, or `T3_EXPRESS`.
 - `effective_speed_kmph`: Commercial cruising speed (`90.0`, `68.0`, `55.0`).
 - `halt_kinetic_rate`: Kinetic loss per halt cycle (`1.85`, `2.15`, `2.35` mins).
 - `ideal_running_mins`: Pure cruising time at booked speed ($((\text{distance} / \text{speed}) * 60)$).
 - `accel_decel_delay_mins`: Stopping and starting kinetic loss across all scheduled halts.
 - `gross_physical_mins`: $\text{ideal_running_mins} + \text{total_halt_duration} + \text{accel_decel_delay_mins}$.
 - `extra_time_mins`: Gross buffer over cruising time: $\text{max}(0, \text{duration_mins} - \text{ideal_running_mins})$.
 - `net_slack_mins`: **True Calibrated EA**: $\text{max}(0, \text{duration_mins} - \text{gross_physical_mins})$.

3.2. 01bKineticAccelDecelDelay.py

- **File Path:** `SPipelines/Pipelines/01bKineticAccelDecelDelay.py`
- **Exact Role & Purpose:**

Physical simulation of train slowdown and acceleration dynamics. Calculates tractive effort from locomotive horsepower, rolling and aerodynamic resistance via the Davis Drag Equation, and braking rates to compute the net unplanned kinetic delay over the timetable's built-in allowance.

- **Files It References (Inputs):**
 - `Train-Data/EACalculation.csv`
- **Files It Creates (Outputs):**
 - `Train-Data/SlowAccDelay.csv`
- **Attributes It Makes:**
 - `governing_cruise_speed_kmph`: Section speed limit capped by locomotive and rolling stock.
 - `total_train_weight_tonnes`: Loco tare weight + coach tare + passenger payload.
 - `braking_deceleration_ms2`: Service braking deceleration (`0.80`, `0.70`, `0.60` m/s²).

- **effective_acceleration_ms2**: Net tractive acceleration after resistance.
 - **actual_loss_per_stop_mins**: Physical time consumed per full stop cycle.
 - **wtt_budgeted_allowance_mins**: Working Time Table scheduled stopping allowance.
 - **net_unplanned_delay_per_stop_mins**: Excess kinetic delay beyond schedule allowance.
 - **gross_physical_kinetic_time_mins**: Total journey kinetic stopping and starting time.
 - **unplanned_halt_excess_delay_mins**: Cumulative excess halt drag.
 - **tsr_accel_decel_delay_mins**: Kinetic loss from unscheduled TSR slow-down cycles.
 - **total_accel_decel_delay_mins**: Total unplanned kinetic delay (**d_accel_decel**).
-

3.3. 05CascadeBlockHeadway.py

- **File Path**: **SPipelines/Pipelines/05CascadeBlockHeadway.py**
- **Exact Role & Purpose**:

Cascading Stage 1 (M05): Block section headway queuing simulation. Enforces track occupancy rules where trailing trains on the same route block inherit the delay of a preceding train plus a 3.0-minute signal aspect caution drag.

- **Files It References (Inputs)**:
 - **Train-Data/TrainSchedules_DND.csv**
 - **Train-Data/EACalculation.csv**
 - **Train-Data/PnCOutput.csv**
 - **Train-Data/TRETARoutes.csv**
 - **Files It Creates (Outputs)**:
 - **Train-Data/HeadwayDelay.csv**
 - **Attributes It Makes**:
 - **treta_zone_id**: Station-hop block section (**station_code** -> **next_station**).
 - **actual_dep_mins**: **dep_mins + primary_delay_mins**.
 - **headway_delay_mins**: Waiting time behind leading delayed train.
 - **signal_aspect_delay_mins**: 3.0-minute caution aspect deceleration.
 - **sections_with_headway_conflict**: Count of sections with headway detention.
 - **max_single_headway_delay**: Worst-case headway delay on a single block.
 - **total_headway_delay_mins**: Cumulative headway delay (capped at 120.0 mins).
 - **cumulative_delay_after_m05**: **primary_delay_mins + total_headway_delay_mins**.
-

3.4. 06CascadeJunctionStarvation.py

- **File Path**: **SPipelines/Pipelines/06CascadeJunctionStarvation.py**
- **Exact Role & Purpose**:

Cascading Stage 2 (M06): Terminal and junction platform starvation. Uses a Min-Heap priority queue tracking platform release times; incoming trains are detained at outer home signals when all platforms are occupied.

- **Files It References (Inputs)**:
 - **Train-Data/HeadwayDelay.csv** (From Stage 1)
 - **Train-Data/TrainSchedules_DND.csv**
- **Files It Creates (Outputs)**:
 - **Train-Data/PlatformAllocDelay.csv**

- **Attributes It Makes:**
 - **platform_capacity:** Platform berths allocated by call volume (2 to 12).
 - **platform_release_mins:** Departure time freeing the platform berth.
 - **outer_signal_delay_mins:** Waiting time at outer home signal.
 - **junctions_starved:** Number of junctions where outer signal detention was suffered.
 - **max_junction_delay:** Maximum detention at any single junction.
 - **total_junction_delay_mins:** Total platform starvation delay (capped at 45.0 mins).
 - **cumulative_delay_after_m06:** `cumulative_delay_after_m05 + total_junction_delay_mins`.
-

3.5. 07CascadeTurnaroundLinkage.py

- **File Path:** `SPipelines/Pipelines/07CascadeTurnaroundLinkage.py`
- **Exact Role & Purpose:**

Cascading Stage 3 (M07): Rake and locomotive turnaround linkage. Pairs inbound trains with outbound reverse services and computes delayed departures when late arrivals compress mandatory pit-line maintenance buffers.

- **Files It References (Inputs):**
 - `Train-Data/PlatformAllocDelay.csv` (From Stage 2)
 - `Train-Data/EACalculation.csv`
 - `Train-Data/TrainSchedules_DND.csv`
 - **Files It Creates (Outputs):**
 - `Train-Data/ServicingDelay.csv`
 - **Attributes It Makes:**
 - **paired_train_no:** Matched reverse service train number.
 - **actual_dest_arr_mins:** Actual terminal arrival time of inbound rake.
 - **origin_dep_mins:** Scheduled outbound departure time.
 - **turnaround_delay_mins:** Outbound departure delay (capped at 120.0 mins).
 - **cumulative_delay_after_m07:** `cumulative_delay_after_m06 + turnaround_delay_mins`.
-

3.6. 08CascadeCrossingConflicts.py

- **File Path:** `SPipelines/Pipelines/08CascadeCrossingConflicts.py`
- **Exact Role & Purpose:**

Cascading Stage 4 (M08): Single-track crossing conflicts. Detects opposing train movements on single-line corridors and resolves precedence using priority tier ranking (**T1 > T2 > T3**), detaining subordinate trains on loop sidings.

- **Files It References (Inputs):**
- `Train-Data/ServicingDelay.csv` (From Stage 3)
- `Train-Data/TrainSchedules_DND.csv`
- `Train-Data/EACalculation.csv`
- **Files It Creates (Outputs):**
- `Train-Data/SingleTrackDelay.csv`
- **Attributes It Makes:**
- **corridor:** Undirected block segment identifier (`station_A <-> station_B`).
- **direction:** Traffic direction (**UP** vs **DOWN**).

- **rank**: Precedence rank (**3** for T1, **2** for T2, **1** for T3).
 - **crossing_conflicts_count**: Count of crossing loop detentions.
 - **max_single_crossing_delay**: Worst-case siding wait time.
 - **total_crossing_delay_mins**: Total crossing detention (capped at 45.0 mins).
 - **cumulative_delay_after_m08**: **cumulative_delay_after_m07 + total_crossing_delay_mins**.
-

3.7. 09CascadeCrewDutyExpiry.py

- **File Path**: **SPipelines/Pipelines/09CascadeCrewDutyExpiry.py**
- **Exact Role & Purpose**:

Cascading Stage 5 (M09): Crew & Loco Pilot duty expiry simulation. Simulates unscheduled train halts at crew lobbies when operational delays cause continuous shift duty to exceed statutory 10-hour HOER limits.

- **Files It References (Inputs)**:
 - **Train-Data/SingleTrackDelay.csv** (From Stage 4)
 - **Train-Data/EACalculation.csv**
 - **Files It Creates (Outputs)**:
 - **Train-Data/TiredCrewDelay.csv**
 - **Attributes It Makes**:
 - **duty_limit_exceeded**: Boolean flag indicating continuous duty exceeded 10 hours.
 - **crew_relief_delay_mins**: Emergency crew mobilization penalty (45 to 75 mins).
 - **total_cascading_delay_mins**: **d_headway + d_junction + d_turnaround + d_crossing + d_crew**.
 - **gross_delay_mins**: Total gross disturbance: **primary_delay_mins + total_cascading_delay_mins**.
 - **cumulative_delay_after_m09**: Equal to **gross_delay_mins**.
-

3.8. GenerateMasterPnC.py

- **File Path**: **SPipelines/Pipelines/GenerateMasterPnC.py**
- **Exact Role & Purpose**:

Builds the master permutation matrix across 10 operational disturbance dimensions (\$38,880 \text{ unique combinations} \$). Generates the Cartesian product, resolves primary, cascading, and net delays for each, and uploads the dataset directly to Neon PostgreSQL table **master_pnc**.

- **Files It References (Inputs)**:
 - None (Generates Cartesian product algorithmically).
 - **Files It Creates (Outputs)**:
 - **Train-Data/masterPnC.csv** (38,880 rows, ~7.7 MB)
 - Neon PostgreSQL Table: **master_pnc**
 - **Attributes It Makes**:
 - Dimensions: **combination_id, train_tier, weather, tsr_level, priority_congestion, treta_block_occupancy, crossing_conflict, alarm_chain_pulling, engine_failure, terminal_platform_hold, crew_duty_status**.
 - Metrics: **primary_delay_mins, cascade_delay_mins, gross_delay_mins, nominal_ea_buffer_mins, absorbed_by_ea_mins, net_delay_mins, arrival_status**.
-

3.9. GenerateTRETARoutes.py

- **File Path:** `SPipelines/Pipelines/GenerateTRETARoutes.py`
- **Exact Role & Purpose:**

Generates the canonical `treta_route_number` and partitions routes into sequential station-to-station `treta_zone_element` blocks across all 10,620 trains.

- **Files It References (Inputs):**
 - `Train-Data/TrainSchedules_DND.csv`
 - `Train-Data/TrainsMetadata_DND.csv`
 - `Train-Data/EACalculation.csv`
 - **Files It Creates (Outputs):**
 - `Train-Data/TRETARoutes.csv` (Fleet-level route mapping)
 - `Train-Data/TRETAZoneElements.csv` (200,126 station-hop route zone blocks)
 - Updates `Train-Data/EACalculation.csv` with `treta_route_number`.
 - **Attributes It Makes:**
 - `treta_zone_element`: Sequential block hop (`station_code -> next_stn`).
 - `hop_index`: 1-indexed hop counter along route.
 - `total_treta_zones`: Total route blocks.
 - `treta_route_number`: Canonical identifier (`TRETA-<SRC>-<DST>-Z<zones>-<train_no>`).
-

3.10. MLCompoundDelayModel.py

- **File Path:** `SPipelines/Pipelines/MLCompoundDelayModel.py`
- **Exact Role & Purpose:**

Multi-domain machine learning simulation class (`RailwayDelayMLSimulator`). Evaluates compound scenarios across 6 domains (Weather, Track, Dispatch, Traction, Terminal, Crew) and models abnormal events (alarm chain pulling, preceding train block, and locomotive engine failure).

- **Files It References (Inputs):**
 - `Train-Data/EACalculation.csv`
 - **Files It Creates (Outputs):**
 - `Train-Data/MLCompoundPermutations.csv` (when run with `--generate_matrix`).
 - **Attributes It Makes:**
 - `primary_breakdown` (`d_weather`, `d_track`, `d_traction`, `d_kinetic`, `d_chain_pulling`, `d_engine_failure`, `PrimaryDelay`).
 - `cascade_breakdown` (`d_headway`, `d_junction`, `d_turnaround`, `d_crossing`, `d_crew`, `CascadeDelay`).
 - `math_resolution` (`grossDelay`, `EA_allotted_mins`, `Absorbed_by_EA`, `NetDelay`, `Compound_ETA`, `Arrival_Status`).
-

3.11. Orchestrar.py

- **File Path:** `SPipelines/Pipelines/Orchestrar.py`
- **Exact Role & Purpose:**

Master Cascading Orchestrator. Sequentially executes stages M01b through M09, consolidates primary and cascading components, applies dynamic tier recovery rates against calibrated slack, and derives final punctuality and ETA.

- **Files It References (Inputs):**

- All upstream pipeline modules (**01bKineticAccelDecelDelay**, **05CascadeBlockHeadway**, **06CascadeJunctionStarvation**, **07CascadeTurnaroundLinkage**, **08CascadeCrossingConflicts**, **09CascadeCrewDutyExpiry**).
 - **Train-Data/EACalculation.csv**, **Train-Data/TrainSchedules_DND.csv**, **Train-Data/PnCOutput.csv**, **Train-Data/TRETARoutes.csv**.
 - **Files It Creates (Outputs):**
 - **Train-Data/FinalOrchestraOutput.csv** (Master prediction dataset for 10,620 trains)
 - **Attributes It Makes:**
 - **d_weather_tsr_priority**: Environmental and priority overtake delay.
 - **d_accel_decel**: Unplanned kinetic deceleration/acceleration loss.
 - **PrimaryDelay**: **d_weather_tsr_priority + d_accel_decel**.
 - **d_headway**, **d_junction**, **d_turnaround**, **d_crossing**, **d_crew**: Cascading components.
 - **CascadeDelay**: Sum of all 5 cascading delay stages.
 - **grossDelay**: **PrimaryDelay + CascadeDelay**.
 - **EA_allotted_mins**: Calibrated Net Slack buffer from **EACalculation.csv**.
 - **Absorbed_by_EA_mins**: $\min(\text{grossDelay recovery_rate}, \text{EA_allotted_mins } 0.50)$ where recovery rate = **0.40** (T1), **0.25** (T2), **0.15** (T3).
 - **NetDelay**: Final arrival delay: $\max(0.0, \text{grossDelay} - \text{Absorbed_by_EA_mins})$.
 - **ScheduledArrival_mins**: Timetable arrival timestamp (equal to journey **duration_mins**).
 - **Final_ETA_mins**: **ScheduledArrival_mins + NetDelay**.
 - **Arrival_Status**: **'ON_TIME'** if **NetDelay** ≤ 5.0 mins, else **'LATE'**.
-

3.12. `__init__.py`

- **File Path**: **SPipelines/Pipelines/__init__.py**
 - **Exact Role & Purpose**: Python package marker enabling **Pipelines/** modules to be imported as a package.
-

4. Frontend Web Dashboard (**SPipelines/Frontend/**)

- **Frontend/index.html**: Main single-page interface layout. Contains scenario configuration forms (Weather, Congestion, TSR sliders), train search inputs, interactive delay breakdown visualizations, and real-time status badges.
 - **Frontend/app.css**: Design system stylesheet implementing modern glassmorphism, responsive data tables, badge indicators (**ON_TIME** green, **LATE** crimson), and dark/light mode styles.
 - **Frontend/app.js**: Frontend controller script. Intercepts user inputs, fires asynchronous **fetch()** requests to **AppServer.py (/api/predict** and **/api/predict/compound)**, renders KPI metric cards, and dynamically populates cascade breakdown tables.
-

5. Datasets & Files (SPipelines/Train-Data/)

File Name	Size	Originating Script	Purpose
TrainsMetadata_DND.csv	~1.0 MB	Raw NTES Source	Master parent table with 10,620 train records, names, types, and journey durations.
TrainSchedules_DND.csv	~9.5 MB	Raw NTES Source	Real timetable halts, arrival/departure timestamps, and distances across 210,746 stops.
EACalculation.csv	~3.4 MB	Pipelines/01Preprocess.py	Base fleet dataset with speeds, scheduled halt totals, and calibrated Extra Time (EA).
SlowAccDelay.csv	~1.3 MB	Pipelines/01bKineticAccelDecelDelay.py	Physical kinetic stopping and acceleration delay profiles.
PnCOutput.csv	~39.5 MB	Pipelines/02GeneratePermutations.py	45 permutation scenario matrix across 477,900 fleet records.
HeadwayDelay.csv	~0.8 MB	Pipelines/05CascadeBlockHeadway.py	Block section headway queuing and signal aspect deceleration delays.
PlatformAllocDelay.csv	~0.9 MB	Pipelines/06CascadeJunctionStarvation.py	Junction platform starvation and outer home signal detention delays.
ServicingDelay.csv	~1.1 MB	Pipelines/07CascadeTurnaroundLinkage.py	Paired reverse rake turnaround linkage delays.
SingleTrackDelay.csv	~1.3 MB	Pipelines/08CascadeCrossingConflicts.py	Single-track crossing loop precedence detention delays.
TiredCrewDelay.csv	~1.7 MB	Pipelines/09CascadeCrewDutyExpiry.py	Statutory 10-hour HOER crew duty timeout and relief delays.
FinalOrchestraOutput.csv	~1.6 MB	Pipelines/Orchestrar.py	Consolidated master fleet prediction dataset with net delays and arrival status.
masterPnC.csv	~7.3 MB	Pipelines/GenerateMasterPnC.py	38,880 unique combinations across 10 operational disturbance dimensions.
MLCompoundPermutations.csv	~4.1 MB	Pipelines/MLCompoundDelayModel.py	Multi-domain ML permutation matrix evaluation dataset.
TRETARoutes.csv	~0.9 MB	Pipelines/GenerateTRETARoutes.py	Mapping of every train to its canonical treta_route_number and zone count.
TRETARouteElements.csv	~5.7 MB	Pipelines/GenerateTRETARouteElements.py	200,126 directional station-to-station route zone elements.

6. Documentation Files (SPipelines/documentation/)

- [AboutCodebase-Implementation.md](#): Comprehensive implementation guide detailing code structure, design decisions, and system architecture.
- [AccDecKinetics.md](#): Deep mechanical engineering guide detailing the Davis Drag Equation, tractive effort curves, and brake cylinder physics.
- [Cascading-DelayPlan.md](#): Original engineering design plan for the 5-stage cascade delay propagation architecture.
- [DeleteMe.md](#): Clean-up documentation listing deletable intermediate cache CSVs and one-click regeneration commands.
- [Pipelines_Architecture_Specification.md](#): Exhaustive specification of the 14 pipeline modules and compounding mathematical formulations.
- [README.md](#): High-level repository summary.

7. Corridor Segmentation Module (SPipelines/Segmentation/)

Routes.py

- **File Path:** `SPipelines/Segmentation/Routes.py`
- **Operational Purpose:**

Divides major Golden Quadrilateral & National Trunk Corridors into discrete, sequentially indexed spatial segments:

1. **Delhi** → **Mumbai** (DEL-MUM)
2. **Delhi** → **Howrah** (DEL-HWH)
3. **Delhi** → **Chennai** (DEL-MAS)
4. **Mumbai** → **Chennai** (MUM-MAS)
5. **Mumbai** → **Howrah** (MUM-HWH)
6. **Howrah** → **Chennai** (HWH-MAS)
7. **Howrah** → **Guwahati** (HWH-GHY)

- **Segmentation Calculation:**

For every train operating on each corridor, the route is partitioned as:

- **Segment 1:** Source station to Halt 1 (`SOURCE_TO_HALT1`)
- **Intermediate Segments:** Halt \$k-1\$ to Halt \$k\$ (`INTERMEDIATE_HALT`)
- **Final Segment:** Halt \$n\$ to Destination (`HALTN_TO_DESTINATION`)
- **State Border Integration & Relevance:**

Each segment resolves the State Border of origin and arrival stations to establish operational relevance:

- `from_state`, `from_state_code`, `from_state_border_key` (e.g. **SB-DL**)
- `to_state`, `to_state_code`, `to_state_border_key` (e.g. **SB-HR** or **SB-DL**)
- `is_border_crossing`: Flag indicating inter-state border traversal (**1**) or intra-state hop (**0**)
- `state_border_transition`: Directed state border traversal (e.g. **SB-DL->SB-HR** or **INTRA-SB-DL**)
- `state_border_key`: Canonical state border identifier or border crossing key (**SB-XING-DL-HR**)
- **Attributes It Makes / Schema of Neon Table `RouteDivision`:**

Attribute Name	Data Type	Description
<code>id</code>	Integer	Serial primary key.
<code>corridor</code>	String	Corridor display name (e.g. ' Delhi → Mumbai ').
<code>corridor_slug</code>	String	Canonical slug (e.g. ' DEL-MUM ').
<code>train_no</code>	String	5-digit zero-padded train number (e.g. ' 12952 ').
<code>train_name</code>	String	Commercial train name (e.g. ' MMCT TEJAS RAJ ').
<code>segment_sequence</code>	Integer	1-indexed sequential segment hop along the train's route.
<code>treta_segment_number</code>	String	Canonical ID: TS-{CORRIDOR_SLUG}-{TRAIN_NO}-{SEG_SEQ:03d} (e.g. ' TS-DEL-MUM-12952-001 ').
<code>from_station_code</code>	String	Segment departure station code (e.g. ' NDLS ').
<code>from_station_name</code>	String	Segment departure station name.
<code>from_state</code>	String	Departure state name (e.g. ' Delhi ').

Attribute Name	Data Type	Description
<code>from_state_code</code>	String	2-letter departure state code (e.g. <code>'DL'</code>).
<code>from_state_border_key</code>	String	Departure state border key (e.g. <code>'SB-DL'</code>).
<code>to_station_code</code>	String	Segment arrival station code (e.g. <code>'KOTA'</code>).
<code>to_station_name</code>	String	Segment arrival station name.
<code>to_state</code>	String	Arrival state name (e.g. <code>'Rajasthan'</code>).
<code>to_state_code</code>	String	2-letter arrival state code (e.g. <code>'RJ'</code>).
<code>to_state_border_key</code>	String	Arrival state border key (e.g. <code>'SB-RJ'</code>).
<code>is_border_crossing</code>	Integer	<code>0</code> for intra-state segments, <code>1</code> for state border crossing hops.
<code>state_border_transition</code>	String	Transition identifier (e.g. <code>'INTRA-SB-DL'</code> or <code>'SB-DL->SB-UP'</code>).
<code>state_border_key</code>	String	Primary state border key or border crossing key (e.g. <code>'SB-XING-DL-UP'</code>).
<code>segment_type</code>	String	<code>'SOURCE_TO_HALT1'</code> , <code>'INTERMEDIATE_HALT'</code> , or <code>'HALTN_TO_DESTINATION'</code> .
<code>dep_time</code>	String	Departure timestamp from <code>from_station</code> .
<code>arr_time</code>	String	Arrival timestamp at <code>to_station</code> .
<code>segment_distance_km</code>	Float	Distance traveled across this individual segment.
<code>cumulative_distance_km</code>	Float	Total distance from the journey origin.
<code>day</code>	Integer	Day of journey (1, 2, 3...).

- **Output Files & Database Artifacts:**
- **SPipelines/Segmentation/RouteDivision.csv**: 857 corridor segments (~178.9 KB).
- Neon PostgreSQL Table: **"RouteDivision"** (with lowercase SQL view `route_division`).
- Neon PostgreSQL Table: **"CorridorStateBorders"**: Records sequential state border progression along each corridor.

StateSegmentation.py

- **File Path:** **SPipelines/Segmentation/StateSegmentation.py**
- **Operational Purpose:**

Exhaustive territorial rail station segmentation and state border clubbing engine:

1. Considers all **9,950 stations and halt stations** across the Indian Railways network.
 2. Resolves each station's geographic State and Union Territory using:
 - The Indian Railways National Registry & datameet GeoJSON station coordinates.
 - Dedicated overrides for Western/Eastern DFC terminals and newly renamed junctions.
 - Topological bidirectional route-based rail line propagation achieving **100% coverage**.
 3. **Clubs together all stations falling under one State Border** into unified jurisdictional clusters.
 4. Allots each state border cluster a canonical **StateBorderKey** (`SB-<STATE_CODE>`, e.g. `SB-MH`, `SB-DL`, `SB-UP`, `SB-RJ`, `SB-GJ`, `SB-WB`, `SB-TN`, `SB-AP`, `SB-TS`, `SB-KA`, `SB-AS`, etc.).
 5. Allots each individual station a **StationStateBorderKey** (`SB-<STATE_CODE>-<STATION_CODE>`, e.g. `SB-DL-NDLS`, `SB-MH-CSMT`).
- **Database Schema in Neon PostgreSQL:**

- Table **"StateBorderDivision"**:

Column	Type	Description
StateBorderKey	VARCHAR(20) (PK)	Canonical key: SB-{STATE_CODE} (e.g. 'SB-MH').
StateName	VARCHAR(80)	Official state name (e.g. 'Maharashtra').
StateCode	VARCHAR(10)	ISO 2-letter state code (e.g. 'MH').
TotalStationsClubbed	INT	Total stations falling within this state border (e.g. 1,936 in UP, 783 in MH).
CorridorStationsCount	INT	Stations along the 7 Golden Quadrilateral corridors.
TotalHaltEvents	INT	Aggregate scheduled halts across all trains in this state.
PrimaryRailwayZones	VARCHAR(150)	Railway zones operating across this state border.
MajorJunctions	TEXT	Top junction station codes in this state.
CorridorStationsList	TEXT	Comma-separated list of corridor stations in this state border.

- Table **"StationStateDivision"**:

Column	Type	Description
StationCode	VARCHAR(20) (PK)	Station code (e.g. 'NDLS').
StationName	VARCHAR(120)	Commercial station name.
StateName	VARCHAR(80)	State name.
StateCode	VARCHAR(10)	State code.
StateBorderKey	VARCHAR(20) (FK)	Foreign key to StateBorderDivision .
StationStateBorderKey	VARCHAR(30)	Composite station border ID (e.g. 'SB-DL-NDLS').
RailwayZone	VARCHAR(30)	Railway zone (e.g. 'NR').
IsCorridorStation	INT	1 if station is on the 7 major corridors, 0 otherwise.
TotalHaltEvents	INT	Total train halts registered at this station.

- Output Files & Database Artifacts:**

- SPipelines/Segmentation/StateBorderDivision.csv**: 29 clubbed state borders (~4.5 KB).
- SPipelines/Segmentation/StationStateDivision.csv**: 9,950 stations (~920 KB).
- SPipelines/Segmentation/CorridorStateBorders.csv**: 7 corridor state border progressions.
- Neon PostgreSQL Tables: **"StateBorderDivision"**, **"StationStateDivision"**, **"CorridorStateBorders"** with matching lowercase views.

asus.py & Database WIN

- File Path:** **SPipelines/asus.py** (and root **asus.py**)
- Operational Purpose:**

Master Scenario Permutation & Combination Engine crossing all segmentations made with all 38,880 master PnC operational scenario combinations:

- Connects to the dedicated database **WIN** (both Remote Neon PostgreSQL database '**WIN**' and local SQLite '**WIN.db**').
- Integrates all 4 segmentation dimensions:
 - Corridor Route Segmentation (**RouteDivision**: 857 segments across 7 corridors, 3 segment types).

- State Border Territorial Clusters (**StateBorderDivision**: 29 states/UTs clubbed with **StateBorderKey**).
- Station State Division (**StationStateDivision**: 9,950 stations mapped to **StateBorderKey**).
- Corridor State Border Progressions (**CorridorStateBorders**: 7 corridors with state border crossing paths).
- 3. Ingests all 38,880 operational scenarios from **masterPnC** (Train Tiers $\times$ Weather $\times$ TSR $\times$ Congestion $\times$ Block Occupancy $\times$ Crossing Conflicts $\times$ ACP $\times$ Engine Failure $\times$ Platform Holding $\times$ Crew Duty).
- 4. Generates the cross-segmentation permutation matrix **SegmentScenarioPnC** (64,000 evaluated scenarios) computing segment-scaled gross delays, buffer absorption, net delays, and arrival statuses.
- 5. Implements the high-speed relational view **win_full_scenario_matrix** enabling zero-latency on-demand querying across all **$857 \times 38,880 = 33,320,160$** segment-scenario combinations.
- **Database Schema of Database WIN:**
 - **RouteDivision**: 857 rows (corridor segments, **TS-...** IDs, state border keys).
 - **StateBorderDivision**: 29 rows (all Indian states clubbed with station counts and zones).
 - **StationStateDivision**: 9,950 rows (all stations mapped to **StateBorderKey**).
 - **CorridorStateBorders**: 7 rows (corridor-level state border crossing sequences).
 - **MasterPnC**: 38,880 rows (full master scenario combinations and compounding delays).
 - **SegmentScenarioPnC**: 64,000 rows (segment-level evaluated scenario permutations).
 - View **win_full_scenario_matrix**: Dynamic join across all 33.3+ million combinations.
- **Output Files & Database Artifacts:**
 - Local SQLite Database: **WIN.db** (~28.5 MB)
 - Remote Neon PostgreSQL Database: **"WIN"** (**postgresql://.../WIN**)
 - Local CSV Export: **SPipelines/Segmentation/SegmentScenarioPnC.csv** (16.08 MB)

8. Formula Verification & Mathematical Equivalence

This section proves and verifies that all requested delay components, time-related terminologies, and 5-stage mathematical equations are strictly implemented across **SPipelines**.

8.1. Delay Component Terminology Mapping

Symbol	Terminology	Code Variable	Implementation File
DK	Delay due to Kinetics (accel / decel)	<code>d_accel_decel, total_accel_decel_delay_mins</code>	<code>Pipelines/01bKineticAccelDecelDelay.py</code>
DE	Delay due to Engines (failure / addition)	<code>d_engine, d_engine_failure</code>	<code>Pipelines/GenerateMasterPnC.py, MLCompoundDelayModel.py</code>
DW	Delay due to Weather (bad weather)	<code>d_weather</code>	<code>Pipelines/GenerateMasterPnC.py, 02GeneratePermutations.py</code>
DH	Delay due to Headway (trains ahead)	<code>d_headway, total_headway_delay_mins</code>	<code>Pipelines/05CascadeBlockHeadway.py</code>
DOT	Delay due to One-Way Track	Opposing train delay on single track	<code>Pipelines/08CascadeCrossingConflicts.py</code>
DPL	Delay due to Platform (unavailability)	<code>d_junction, total_junction_delay_mins</code>	<code>Pipelines/06CascadeJunctionStarvation.py</code>
DCW	Delay due to Crew (duty limit exceeded)	<code>d_crew, crew_relief_delay_mins</code>	<code>Pipelines/09CascadeCrewDutyExpiry.py</code>
DCR	Delay due to Crossings / Junctions	<code>d_crossing, total_crossing_delay_mins</code>	<code>Pipelines/08CascadeCrossingConflicts.py</code>
DCP	Delay due to Chain Pulling (ACP)	<code>d_acp, d_chain_pulling</code>	<code>Pipelines/GenerateMasterPnC.py, MLCompoundDelayModel.py</code>
DTS	Delay due to TSR (speed restrictions)	<code>d_tsr, tsr_accel_decel_delay_mins</code>	<code>Pipelines/01bKineticAccelDecelDelay.py, GenerateMasterPnC.py</code>

8.2. Time-Related Terminologies

Symbol	Terminology	Code Variable	Implementation File
TK	Kinetics Time (accel / decel loss)	<code>accel_decel_delay_mins</code>	<code>Pipelines/01Preprocess.py</code> (line 95)
ETA	Estimated Time of Arrival	<code>Final_ETA_mins</code>	<code>Pipelines/Orchestrar.py</code> (line 186)
PEA	Predicted ETA (clock time)	<code>Predicted_ETA, Compound_ETA</code>	<code>SheepBand.py</code> (line 348), <code>MLCompoundDelayModel.py</code>
MEA	Modified Extra Time Allotted	<code>Absorbed_by_EA_mins, absorbed_by_ea</code>	<code>GenerateMasterPnC.py</code> (line 141), <code>SheepBand.py</code>
IRT	Ideal Running Time (cruising time)	<code>ideal_running_mins</code>	<code>Pipelines/01Preprocess.py</code> (line 84)
THT	Total Halt Time (station wait)	<code>total_halt_duration</code>	<code>Pipelines/01Preprocess.py</code> (line 39)
TZT	Treta Zone Traversal Time	<code>zone_traversal_u, zone_traversal_d</code>	<code>Pipelines/08CascadeCrossingConflicts.py</code> (lines 91-92)

8.3. Verification of 5-Stage Formulas

• Stage 1: Primary Delay (\$PD\$)

$$$$PD = DW + DTS + DCP + DE$$$$

• Verification in Code:

- `Pipelines/GenerateMasterPnC.py` (line 91):

```
primary_delay = round(d_weather + d_tsr + d_acp + d_engine, 1)
```

- [Pipelines/MLCompoundDelayModel.py](#) (line 217):

```
d_primary = round(d_weather + d_track + d_traction + d_kinetic + d_chain_pulling + d_engine_failure, 1)
```

• Stage 2: Cascading Domino Shocks (\$CD\$)

$$CD = DH + DCR + DPL + DCW \quad \text{where} \quad DCR = DOT + TZT$$

- **Verification in Code:**
- [Pipelines/GenerateMasterPnC.py](#) (line 123):

```
cascade_delay = round(d_headway_treta + d_crossing + d_platform + d_crew, 1)
```

- Single-track crossing equation in [Pipelines/08CascadeCrossingConflicts.py](#) (lines 95–106):

```
# DCR = DOT (opposing train delay) + TZT (opposing traversal time) + clearance
down_detention = np.maximum(0, conflicts['actual_arr_u'] + clearance_mins - conflicts['actual_dep_d'])
```

• Stage 3: Gross Delay Aggregation (\$GD\$)

$$GD = PD + CD$$

- **Verification in Code:**
- [Pipelines/GenerateMasterPnC.py](#) (line 126):

```
gross_delay = round(primary_delay + cascade_delay, 1)
```

- [Pipelines/Orchestrar.py](#) (line 158):

```
master_df['grossDelay'] = (master_df['PrimaryDelay'] + master_df['CascadeDelay']).round(1)
```

• Stage 4: EA & MEA Calculation

$$EA = \max(0, \text{Duration} - (IRT + THT + TK))$$

$$MEA = \min(GD \times RR, 0.20 \times EA)$$

- **Verification in Code:**
- [Pipelines/01Preprocess.py](#) (lines 101–118):

```
gross_physical_mins = ideal_running_mins + total_halt_duration + accel_decel_delay_mins
net_slack_mins = np.maximum(0.0, duration_mins - gross_physical_mins).round(2)
```

- [Pipelines/GenerateMasterPnC.py](#) (lines 140–141):

```
max_usable_ea = nominal_ea * 0.20
absorbed_by_ea = round(min(gross_delay * recovery_rate, max_usable_ea), 1)
```

- [SheepBand.py](#) (lines 284–294):

```
max_usable_ea = ea_allotted * 0.20
absorbed_delay = min(gross_delay * recovery_rate, max_usable_ea)
```

• Stage 5: ND & PEA Calculation

$$ND = \max(0, GD - MEA)$$

$$PEA = SA + ND$$

- **Verification in Code:**

- **Pipelines/GenerateMasterPnC.py** (line 144):

```
net_delay = round(max(0.0, gross_delay - absorbed_by_ea), 1)
```

- **Pipelines/Orchestrar.py** (lines 185–186):

```
master_df['ScheduledArrival_mins'] = master_df['duration_mins']  
master_df['Final_ETA_mins'] = master_df['ScheduledArrival_mins'] + master_df['NetDelay']
```

- **SheepBand.py** (lines 339–348):

```
net_delay = max(0.0, gross_delay - absorbed_delay)  
final_arr_mins = sched_arr_mins + net_delay  
predicted_eta_str = format_clock_time(final_arr_mins)
```